

Buổi 11 — RESTful API với ASP.NET Core, EF Core và Swagger

Môn: Lập trình Web

Thời lượng: 3 giờ

Giảng viên: _____

Mục tiêu buổi học

Kết thúc buổi học, sinh viên có thể:

1. Giải thích **REST**, **HTTP method** và các **mã trạng thái** thường dùng.
2. Xây dựng **API Controller** với attribute routing, đặt cùng ứng dụng MVC.
3. Cài đặt **CRUD API** bằng EF Core, có **DTO** và validation.
4. Kiểm thử API bằng **Swagger UI** và trả đúng mã trạng thái.

Lộ trình buổi học

Phần	Nội dung	Thời lượng
1	REST, HTTP method, mã trạng thái	25 phút
2	API Controller + DTO	30 phút
3	CRUD API với EF Core	30 phút
4	Swagger + bảo mật API	20 phút
5	Live coding API posts + kiểm thử Swagger	40 phút
6	Thực hành + tổng kết dự án	30 phút

Ôn tập nhanh buổi trước

Trả lời nhanh:

1. `Skip / Take` dùng để làm gì? (Buổi 6)
2. `[Authorize(Roles="Admin")]` có tác dụng gì? (Buổi 7)
3. `ViewModel/DTO` tách dữ liệu khỏi `Entity` để làm gì? (Buổi 5)

Câu hỏi mở: một **app di động** muốn lấy danh sách bài viết thì cần gì?

Bối cảnh: vì sao cần API?

Đến giờ `BlogManager` chỉ phục vụ qua **HTML** (Razor). Nhưng dữ liệu còn được dùng bởi:

- Ứng dụng **di động**
- Trang **JavaScript** phía client
- Hệ thống **bên thứ ba**

Chúng không cần HTML mà cần **dữ liệu thuần (JSON)** qua **RESTful API**.

API đặt ở đâu trong monolith?

```
Controllers/  
  PostsController.cs      # MVC/Razor UI → trả HTML  
  Api/  
    PostsApiController.cs  # RESTful API → trả JSON  
Data/                    # ApplicationDbContext dùng chung  
Models/                  # Entity dùng chung  
Dtos/                    # đối tượng truyền dữ liệu cho API
```

API **không** là dự án riêng — cùng ứng dụng, chung DbContext/Entity.

Khác biệt: trả **JSON** thay vì View, routing tiền tố `/api`.

REST và tài nguyên

API tổ chức quanh **tài nguyên** (danh từ); hành động thể hiện bằng **HTTP method**.

Nên dùng	Tránh
GET /api/posts	GET /api/getAllPosts
POST /api/posts	POST /api/createPost
DELETE /api/posts/5	GET /api/deletePost?id=5

URL chỉ **danh từ**; method cho biết **hành động**.

HTTP method

Method	Ý nghĩa	Ví dụ
GET	Lấy dữ liệu (không đổi trạng thái)	GET /api/posts/5
POST	Tạo mới	POST /api/posts
PUT	Cập nhật toàn bộ	PUT /api/posts/5
DELETE	Xóa	DELETE /api/posts/5

Mã trạng thái HTTP

Trả **đúng** mã để chương trình gọi biết kết quả:

Mã	Ý nghĩa	Khi nào
200 OK	Thành công, có dữ liệu	GET/PUT ok
201 Created	Tạo mới thành công	POST
204 No Content	Ok, không nội dung	DELETE, PUT
400 Bad Request	Dữ liệu không hợp lệ	Validation fail
401 Unauthorized	Chưa xác thực	Thiếu/sai đăng nhập
403 Forbidden	Không đủ quyền	Thiếu vai trò
404 Not Found	Không tìm thấy	Id không tồn tại

API Controller + attribute routing

```
using Microsoft.AspNetCore.Mvc;

namespace BlogManager.Controllers.Api;

[ApiController]
[Route("api/posts")]
public class PostsApiController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public PostsApiController(ApplicationDbContext context)
    {
        _context = context;
    }
}
```

- `[ApiController]` → tự trả `400` khi model sai, bind từ body
- `[Route("api/posts")]` → tiền tố URL
- `ControllerBase` → có `Ok()`, `NotFound()`, `CreatedAtAction()` ... (không `View()`)

DTO — Data Transfer Object

```
namespace BlogManager.Dtos;

public class PostDto // trả về client
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Author { get; set; } = string.Empty;
    public DateTime PublishedAt { get; set; }
    public bool IsPublished { get; set; }
}

public class PostCreateDto // client gửi lên
{
    [Required, StringLength(200, MinimumLength = 3)]
    public string Title { get; set; } = string.Empty;
    [Required]
    public string Content { get; set; } = string.Empty;
    public string Author { get; set; } = string.Empty;
}
```

⚠ Không trả thẳng Entity (navigation gây lỗi serialize, lộ trường nhạy cảm).

Validation đặt trên DTO đầu vào — nối tiếp ViewModel (Buổi 5).

CRUD API — GET

```
// GET /api/posts
[HttpGet]
public async Task<ActionResult<IEnumerable<PostDto>>> GetPosts()
{
    var posts = await _context.Posts
        .Select(p => new PostDto { Id = p.Id, Title = p.Title, Author = p.Author,
                                PublishedAt = p.PublishedAt, IsPublished = p.IsPublished })
        .ToListAsync();
    return Ok(posts); // 200
}

// GET /api/posts/5
[HttpGet("{id}")]
public async Task<ActionResult<PostDto>> GetPost(int id)
{
    var p = await _context.Posts.FindAsync(id);
    if (p == null) return NotFound(); // 404
    return Ok(new PostDto { Id = p.Id, Title = p.Title, Author = p.Author,
                            PublishedAt = p.PublishedAt, IsPublished = p.IsPublished });
}
```

CRUD API — POST (tạo mới)

```
// POST /api/posts
[HttpPost]
public async Task<ActionResult<PostDto>> CreatePost(PostCreatedDto dto)
{
    var post = new Post { Title = dto.Title, Content = dto.Content, Author = dto.Author };
    _context.Posts.Add(post);
    await _context.SaveChangesAsync();

    var result = new PostDto { Id = post.Id, Title = post.Title, Author = post.Author,
                             PublishedAt = post.PublishedAt, IsPublished = post.IsPublished };
    return CreatedAtAction(nameof(GetPost), new { id = post.Id }, result); // 201
}
```

- [ApiController] → DTO sai tự trả 400
- CreatedAtAction → 201 kèm header Location trỏ tới tài nguyên mới

CRUD API — PUT & DELETE

```
// PUT /api/posts/5
[HttpPut("{id}")]
public async Task<IActionResult> UpdatePost(int id, PostCreateDto dto)
{
    var post = await _context.Posts.FindAsync(id);
    if (post == null) return NotFound(); // 404
    post.Title = dto.Title; post.Content = dto.Content; post.Author = dto.Author;
    await _context.SaveChangesAsync();
    return NoContent(); // 204
}

// DELETE /api/posts/5
[HttpDelete("{id}")]
public async Task<IActionResult> DeletePost(int id)
{
    var post = await _context.Posts.FindAsync(id);
    if (post == null) return NotFound(); // 404
    _context.Posts.Remove(post);
    await _context.SaveChangesAsync();
    return NoContent(); // 204
}
```

Swagger / OpenAPI

Sinh tài liệu + giao diện thử API ngay trong trình duyệt.

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapControllers(); // ⚠ thiếu dòng này → API 404 mọi route
```

Mở `/swagger` → xem endpoint, gửi thử request, xem response + mã trạng thái.

Bảo mật API (kết nối Buổi 7)

```
[HttpDelete("{id}")]  
[Authorize(Roles = "Admin")]  
public async Task<IActionResult> DeletePost(int id) { ... }
```

- Chưa xác thực → 401
- Đã đăng nhập, thiếu vai trò → 403

API thuần thường dùng **token (JWT)** thay cookie — buổi này chỉ giới thiệu khái niệm, tận dụng Identity đã có.

Live coding — Bước 1–3: Dựng API

Bước 1 — `Program.cs` : `AddSwaggerGen` , `UseSwagger` / `UseSwaggerUI` (Development),
`app.MapControllers()`

Bước 2 — Tạo thư mục `Dtos/` : `PostDto` , `PostCreatedDto`

Bước 3 — `Controllers/Api/PostsApiController.cs` : đủ 5 endpoint CRUD

Controller dùng chung `ApplicationDbContext` với phần MVC — đúng tinh thần monolithic.

Bước 4–5: Kiểm thử Swagger

Chạy `dotnet run`, mở `/swagger` :

- ☐ `GET /api/posts` → **200** + danh sách JSON
- ☐ `POST /api/posts` body hợp lệ → **201**, có header `Location`
- ☐ `POST /api/posts` `Title` rỗng → **400** + thông báo lỗi
- ☐ `GET /api/posts/{id}` id không tồn tại → **404**
- ☐ `DELETE /api/posts/{id}` hợp lệ → **204**

```
git add . && git commit -m "Add RESTful API for posts with DTO, validation and Swagger"
```

Lỗi thường gặp (1/2)

Hiện tượng	Nguyên nhân	Cách xử lý
<code>/swagger</code> trả 404	Thiếu <code>UseSwagger</code> /UI hoặc không Development	Thêm middleware; kiểm tra môi trường
API 404 mọi route	Thiếu <code>app.MapControllers()</code>	Thêm vào pipeline
Serialize lỗi vòng lặp	Trả thẳng Entity có navigation	Dùng DTO
POST không trả 400 khi sai	Thiếu <code>[ApiController]</code> / validation	Gắn <code>[ApiController]</code> + Data Annotation

Lỗi thường gặp (2/2)

Hiện tượng	Nguyên nhân	Cách xử lý
<code>CreatedAtAction</code> lỗi tên action	Tham chiếu sai tên	Dùng <code>nameof(GetPost)</code> + đúng route
PUT trả 200 nhưng không đổi dữ liệu	Quên <code>SaveChangesAsync()</code>	Gọi sau khi sửa
API mở public ngoài ý muốn	Thiếu <code>[Authorize]</code>	Gắn <code>[Authorize]</code> / <code>Roles</code> phù hợp

Thực hành trên lớp — 25 phút

Đề bài: RESTful API cho `posts` :

1. Năm endpoint: `GET` , `GET /{id}` , `POST` , `PUT /{id}` , `DELETE /{id}`
2. Dùng `PostDto` / `PostCreateDto` ; validation `Title` (3–200), `Content` bắt buộc
3. Trả đúng `200` / `201` / `204` / `400` / `404`
4. Kiểm thử toàn bộ bằng Swagger

Đầu ra: ảnh chụp Swagger cho từng thao tác + commit.

Checkpoint — Kiểm tra hiểu bài

1. Vì sao URL REST nên dùng danh từ thay vì động từ?
2. Tạo mới thành công nên trả mã nào, kèm thông tin gì?
3. Vì sao nên dùng DTO thay vì trả thẳng Entity?
4. 401 và 403 khác nhau ở tình huống nào?

Tóm tắt

1. **REST** tổ chức quanh tài nguyên; hành động = HTTP method, không phải động từ URL.
2. **Mã trạng thái** (200/201/204/400/401/403/404) báo kết quả cho chương trình gọi.
3. **API Controller**: `ControllerBase` + `[ApiController]` + routing `/api/...`.
4. **DTO** tách dữ liệu API khỏi Entity; validation đặt trên DTO.
5. API dùng chung **DbContext** với MVC — đúng kiến trúc **monolithic**.
6. **Swagger** sinh tài liệu và kiểm thử API ngay trong trình duyệt.

Bài tập về nhà

Bắt buộc: RESTful API cho `posts` — 5 endpoint, DTO + validation (`Title` 3–200, `Content` bắt buộc), trả đúng `200/201/204/400/404`, kiểm thử bằng Swagger.

Tiêu chí: endpoint trả đúng mã, dùng DTO, chặn dữ liệu sai (400), dữ liệu phản ánh trong SQLite, có commit, nộp ảnh chụp Swagger từng thao tác.

Mở rộng: Thêm tìm kiếm + phân trang cho `GET /api/posts` (`search`, `page`, `pageSize`); bảo vệ `DELETE` bằng `[Authorize(Roles="Admin")]`; thử gọi khi chưa đăng nhập để thấy `401`.

Tự kiểm tra: Trả lời 4 câu hỏi ở slide Checkpoint.

Tổng kết học phần & đồ án cuối kỳ

Toàn dự án BlogManager qua 11 buổi:

môi trường → C# → MVC → SQLite/EF Core → CRUD → quan hệ/tìm kiếm/phân trang → Identity → tổ chức code → upload/bảo mật → deploy → **RESTful API**.

Yêu cầu tối thiểu đồ án cuối kỳ:

dữ liệu SQLite/EF Core · ≥ 1 module CRUD · xác thực + phân quyền · validation · tìm kiếm hoặc phân trang · **một RESTful API** · bản deploy hoạt động.

Tài liệu: ASP.NET Core Web API; Swagger/OpenAPI (Swashbuckle); HTTP status codes.

Mốc mã nguồn: 11-starter · 11-completed